

Linux - Part 2

6-MONTH INDUSTRY MASTERCLASS PATHWAY

An enterprise-grade, comprehensive technical architecture, deployment workflow, security hardening guide, and observability plan. Designed for senior engineering professionals (6+ years experience) striving for Principal SRE and Cloud Architect roles.

CORE CONCEPTS | PRODUCTION HARDENING | 20+ INCIDENT SCENARIOS

Automated SRE Learn System | Generated Pdf Generator

Linux - Advanced Tuning, cgroups v2, & System Hardening (Part 2)

1. Topic Introduction

This section explores the highly advanced system configurations, namespaces, unified cgroups v2 architectures, POSIX capabilities, and syscall sandboxing tools (seccomp filters) used to isolate critical daemons in enterprise environments.

2. Why This Topic is Critical

Default systems run daemons with overly broad execution privileges. Hardening processes at the kernel boundary blocks root-level exploits and restricts access to kernel structures.

3. Advanced SRE Capabilities & Resource Metering

- **POSIX Capabilities**: Replaces standard all-or-nothing root privileges. Processes can be run with specific privileges (e.g. `CAP_NET_BIND_SERVICE` allows binding ports < 1024 without running the process as root).
- **cgroups v2 unified control**: Governs CPU quotas, memory swap limits, block device weights, and dirty memory sync pools.

4. Hardened Security & Isolation Configuration

Below is a hardened seccomp filter setup and Capability assignment workflow:

```
# Assign specific capabilities to a binary instead of running as root
sudo setcap 'cap_net_bind_service=+ep' /opt/apiservice/bin/server

# View assigned capabilities
getcap /opt/apiservice/bin/server
```

5. Security & Kernel System Call Hardening

Seccomp (Secure Computing Mode) enables the kernel to instantly terminate a process if it invokes an unauthorized system call (e.g. preventing a web application from calling `execve` to execute a terminal shell).

Linux - Advanced Systems Hardening (Q21 to Q40)

Q21. How do POSIX Capabilities improve security over traditional root/setuid architectures?

Detailed Answer: Traditional Linux security uses an all-or-nothing model (root or standard user). POSIX Capabilities divide root privileges into distinct units (e.g. `CAP_NET_BIND_SERVICE`, `CAP_SYS_TIME`, `CAP_CHOWN`). This allows a daemon to bind to low ports or modify system times without giving it permissions to read/write all system files.

Production Scenario: Pinning `cap_net_bind_service=+ep` on web server binaries allows them to run entirely under standard, non-privileged usernames, protecting the system from arbitrary root shell spawns if the server code is exploited.

Q22. Explain the structural difference between cgroups v1 and cgroups v2.

Detailed Answer: cgroups v1 utilizes separate controllers (CPU, memory, blkio) organized in independent tree structures, leading to resource tracking mismatch issues. cgroups v2 implements a unified single-tree hierarchy, making resource metrics (like allocating dirty page flushes to their correct container limits) highly consistent.

Production Scenario: Running modern container runtimes (like containerd on Kubernetes) with cgroups v2 ensures accurate OOM and I/O saturation triggers per namespace without host context leaks.

Q23. How do you configure a seccomp syscall filter to deny process execution capabilities?

Detailed Answer: By writing a seccomp-bpf filter that intercepts system call requests at the SCI boundary, immediately returning `SECCOMP_RET_KILL` or `SECCOMP_RET_ERRNO` if a blacklisted call (like `execve`, `fork`, or `sys_ptrace`) is requested.

Production Scenario: Setting seccomp filters on container workloads prevents remote code execution exploits from executing `wget` or shell binaries, isolating the attacker within the memory bounds of the active program loop.

Q24. How do you troubleshoot slab cache leaks?

Detailed Answer: Use `slabtop` or check `/proc/slabinfo`. If files are opened and closed frequently without proper closing handles, kernel structures like `dentry` or `inode_cache` grow persistently, starving active RAM.

Production Scenario: An application leaks socket pointers. Over time, `slabtop` shows `ext4_inode_cache` and `sock_inode_cache` occupying 90% of kernel Slab space, triggering page reclaim thrashing.

Q25. What is the impact of vm.max_map_count on elastic search engines?

Detailed Answer: `vm.max_map_count` defines the limit of virtual memory map zones. Elasticsearch uses `mmap` to store indexes, requiring thousands of map areas.

Production Scenario: If `vm.max_map_count` is not bumped to at least 262144, Elasticsearch daemons will crash on startup with memory allocation failures.

Q26. How do you configure and optimize BBR TCP Congestion Control in Linux?

Detailed Answer: BBR (Bottleneck Bandwidth and RTT) is a modern congestion control algorithm developed by Google. To enable BBR, load the kernel module and tune net.core default queuing discipline to `fq` (Fair Queuing) via sysctl:

```
net.core.default_qdisc = fq
net.ipv4.tcp_congestion_control = bbr
```

Production Scenario: Enabling BBR on high-bandwidth, high-latency cross-region connections reduces packet loss retransmission bottlenecks and boosts transfer throughput by up to 10x.

Q27. Explain TCP socket buffer auto-scaling mechanics under Linux.

Detailed Answer: The kernel dynamically scales TCP read/write buffer thresholds using parameters set in `net.ipv4.tcp\_rmem` and `net.ipv4.tcp\_wmem` (specifying min, default, and max buffer byte sizes).

Production Scenario: On a 10Gbps NIC, if max buffers are capped too low (e.g. 128KB), high latency links stall data streams because the sender exhausts the TCP receive window before receiving packet ACKs.

Q28. What is CPU Affinity and how do you bind production services to specific cores?

Detailed Answer: CPU Affinity binds running application threads to dedicated CPU cores, preventing CFS thread migration overhead. Configure it using `taskset -c <CPU\_LIST> -p <PID>` or via the `CPUAffinity=` directive in systemd unit files.

Production Scenario: A highly loaded Redis instance is bound to physical CPU core 1, eliminating cache invalidation lag and reducing request processing variance to sub-milliseconds.

Q29. When is running 'sysctl -w vm.drop_caches=3' dangerous in production?

Detailed Answer: Setting `vm.drop\_caches=3` instructs the kernel to immediately reclaim all clean page caches, dentries, and inodes in RAM, forcing the system to read all file blocks directly from physical storage.

Production Scenario: Executing this on a high-throughput active database server instantly spikes disk queue utilization to 100% and blocks database queries as the filesystem attempts to rebuild cache blocks.

Q30. How do you simulate high network latency and packet loss for chaos engineering testing?

Detailed Answer: Use the Linux `tc` (Traffic Control) network emulator tool. The command below injects 100ms latency and 5% packet loss on the `eth0` network interface:

```
sudo tc qdisc add dev eth0 root netem delay 100ms loss 5%
```

To remove the constraint: `sudo tc qdisc del dev eth0 root`.

Production Scenario: Running this tool in a staging Kubernetes namespace validates that backend microservices can gracefully trigger connection timeouts and fallback retries.

Q31. What is the SRE impact of net.ipv4.tcp_tw_reuse on high-concurrency servers?

Detailed Answer: In high-traffic environments, client sockets quickly enter the `TIME_WAIT` state (duration 60s) after closing, consuming available source ports. Enabling `net.ipv4.tcp_tw_reuse = 1` allows the kernel to safely reuse active `TIME_WAIT` sockets for outgoing connections.

Production Scenario: Bumping this parameter on backend proxy systems prevents outbound socket pool exhaustion errors (e.g. "Cannot assign requested address") during connection peaks.

Q32. How do you adjust service priority and protect critical daemons against OOM events in systemd?

Detailed Answer: Configure `OOMScoreAdjust=-1000` (to completely exempt the process) or `MemoryLimit=` inside the systemd service configurations.

Production Scenario: Exclude critical proxies and log forwarders from OOM killer triggers, ensuring web application containers are terminated first if host memory is depleted.

Q33. How does the kernel govern shared memory allocations for large databases like PostgreSQL?

Detailed Answer: Through kernel parameters `shmmax` (maximum single shared segment size) and `shmall` (total shared memory pages allocation) managed via `sysctl`.

Production Scenario: A PostgreSQL cluster fails to initialize its memory buffers because `shmmax` is smaller than the configured `shared_buffers` size, requiring an update to `sysctl.conf` to set `shmmax` to 80% of RAM.

Q34. What is CFS Bandwidth Throttling and how does it trigger Kubernetes pod latency?

Detailed Answer: Under Kubernetes, container CPU limits translate to CFS quotas. The kernel CFS scheduler tracks CPU cycles consumed during 100ms periods. If a container exhausts its quota before the period ends, it is throttled until the quota resets.

Production Scenario: A Java app experiences latency spikes even with average CPU usage at 30%. SREs check `/sys/fs/cgroup/cpu/cpu.stat` and find `nr_throttled` count rising, prompting them to increase CPU limits or disable hard CFS quotas.

Q35. How do you diagnose inode exhaustion when a filesystem reports ample free space?

Detailed Answer: Run `df -i` to view the percentage of available filesystem inodes. Ext4 and XFS allocate a fixed number of inodes during formatting; if thousands of tiny files are written, inodes exhaust first.

Production Scenario: An API uploads massive numbers of micro-session temp files. Disk capacity is at 20% but `df -i` shows 100% inode saturation. No new files can be written until temp directories are purged.

Q36. What is PAM (Pluggable Authentication Modules) and how does it restrict system resources?

Detailed Answer: PAM manages system authentication rules and loads security sessions. The resource limits

configured in `/etc/security/limits.conf` (e.g., maximum open files, process limits) are applied during session initialization via the `pam_limits.so` module.

Production Scenario: A newly deployed database user is blocked from scaling active connections until `/etc/security/limits.conf` is updated to define `dbuser hard nofile 65535`.

Q37. What is Slab fragmentation and how does it trigger page allocation failures?

Detailed Answer: The Slab allocator manages small memory pieces for kernel objects. Over time, sparse object allocations leave fragmented spaces. When a system requests contiguous memory, page allocation failures occur despite high total free memory.

Production Scenario: Highly loaded virtual nodes trigger kernel page allocation dumps in `dmesg`, indicating cache memory must be compacted or Slab memory reclaimed.

Q38. How do you use bpftrace to inspect kernel file open calls in real-time?

Detailed Answer: `bpftrace` compiles BPF scripts and attaches them directly to kernel probes. Below is a command tracking which PIDs call `sys_enter_open` system calls:

```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_open { printf("%d: %s opens %s\n", pid, comm, str(args->filename)); }'
```

Production Scenario: Tracing file access patterns of an unknown binary reveals it is continuously accessing config paths, isolating a configuration leak.

Q39. What are the key hardening configurations for SSH service daemons?

Detailed Answer: Edit `/etc/ssh/sshd_config` to enforce:

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
ClientAliveInterval 300
```

Production Scenario: Applying these configurations across VM clusters shuts down automated brute-force password scans.

Q40. How do vm.overcommit_memory modes protect systems against sudden OOM events?

Detailed Answer: `vm.overcommit_memory` controls kernel memory allocations:

- **0 (Heuristic)**: Kernel makes educated guesses based on free margins.
- **1 (Always)**: Allocates all requested memory blocks.
- **2 (Strict)**: Allocation is bounded strictly by swap size + a configured percentage of RAM.

Production Scenario: Setting overcommit to `2` prevents processes from allocation surprises, ensuring databases fail predictably at runtime instead of getting killed suddenly by the OOM killer.